Department of Electronics Engineering

Walchand Institute of Technology, Solapur

Datathon Project Report

(Data Analytics — Sem VI)

ON

IPL Teams & Players Performance Analytics

Submitted by

1. **Parth Kulkarni** **Roll No. 36**
2. **Dhruv Sawarkar** **Roll No. 57**
3. **Onkar Tulshigar** **Roll No. 60**

Department of Electronics Engineering,
Walchand Institute of Technology, Solapur
(*An Autonomous Institute*)
Ashok Chowk, Solapur 413006
YEAR- 2025-26

INDEX

Introduction and Problem Definition

1.1 Background

The Indian Premier League (IPL) is one of the most data-rich cricket tournaments in the world. Since its inception in 2008, every ball, wicket, run, and match outcome has been meticulously recorded. This wealth of structured data makes IPL an ideal candidate for applied data analytics enabling teams, analysts, and fans to derive actionable insights from historical patterns. Sports analytics has grown into a multi-billion-dollar field globally, with franchises using data to guide player auctions, match strategies, and batting orders. This project draws from that tradition, applying academic data analytics techniques to real IPL data spanning 12 seasons and over 179,000 individual deliveries.

1.2 Need and Problem Statement

Despite the availability of rich IPL data, teams and analysts often rely on intuition rather than systematic analysis. This project addresses that gap by building a structured analytics pipeline covering both team-level and player-level dimensions of the game

Problem Statement:

To analyze IPL historical match and ball-by-ball data (2008–2020) to identify key factors that influence match outcomes, predict match-winning probability using machine learning, and segment players into distinct performance archetypes — enabling data-driven insights for team strategy and player selection decisions.

1.3 Objectives

- Integrate concepts of data collection, pre-processing, EDA, modelling, and visualization into one end-to-end pipeline.
- Build a Random Forest classifier to predict match-winning probability based on match-level features.
- Segment IPL batsmen into distinct archetypes using K-Means clustering on batting statistics.
- Identify team performance trends, toss strategy patterns, and over-phase run distributions.
- Communicate all findings through an interactive Power BI dashboard and this structured report.

Dataset Description

The dataset used is the **IPL Complete Dataset (2008–2020)**, sourced from Kaggle. It consists of two inter-related CSV files that together capture every match and every delivery bowled across 12 IPL seasons.

Attribute	matches.csv	deliveries.csv
Source	Kaggle — IPL Complete Dataset	Kaggle — IPL Complete Dataset
Records	756 matches	179,078 deliveries
Seasons	2008 – 2020	2008 – 2020
Key Columns	team1, team2, winner, toss_winner, toss_decision, venue, season, player_of_match	match_id, batting_team, batsman, batsman_runs, bowler, player_dismissed, over
Target Variable	winner (match outcome)	batsman_runs, total_runs

Both files are joined on the **match_id** key.

- *matches.csv* drives team-level analysis and win prediction modelling;
- *deliveries.csv* drives player statistics, over-phase analysis, and clustering feature construction.

Note: Team names were standardised across seasons

- 'Delhi Daredevils' → 'Delhi Capitals',
- 'Deccan Chargers' → 'Sunrisers Hyderabad',
- 'Kings XI Punjab' → 'Punjab Kings'.

These corrections were applied uniformly before any analysis

Data Pre-processing

Raw data was cleaned, standardised, and enriched with engineered features before any modelling or visualisation. The pre-processing pipeline was implemented in Python using pandas and scikit-learn, documented in **data_preprocessing.ipynb**.

Step 1: Missing Value Imputation: Null values in the player_of_match column were filled with the category 'Unknown'. Missing city values were imputed based on information extracted from the respective venue strings.

Step 2: Duplicate Row Elimination: Duplicate rows were identified and removed from both datasets using the pandas.drop_duplicates() method.

Step 3: Team Name Standardisation: The names of four franchises that have been renamed across different seasons were unified to their current identities for consistency in aggregation and group-by operations.

Step 4: Over Phase Categorisation: Each delivery was assigned a temporal phase label: Powerplay (overs 1–6), Middle Overs (7–15), or Death Overs (16–20), to enable phase-wise analysis of run distribution.

Step 5: Feature Engineering:A binary indicator, toss_win_match_win, was created. First-innings runs were aggregated per match. Individual player batting statistics (total runs, strike rate, sixes, and average runs per match) were calculated for player clustering.

Step 6: Label Encoding: Categorical features (team names, venue, and toss_decision) were transformed into integer representations using scikit-learn's LabelEncoder before being input into the Random Forest classification model.

Step 7: Train-Test Dataset Splitting: The match feature table was partitioned into training ($n=604$) and test ($n=152$) sets using an 80/20 stratified split with sklearn.model_selection.train_test_split and a fixed random_state of 42.

Tools Utilised: Python 3.12, pandas 2.1, numpy 1.24, scikit-learn 1.3, matplotlib 3.7, seaborn 0.12, and Power BI Desktop.

Model Building & Evaluation

4.1 Exploratory Data Analysis (EDA)

EDA was performed on both match-level and delivery-level data to uncover patterns before modelling. Six key visualisations are presented below.

Figure 1: IPL Title Wins by Team (2008–2020).

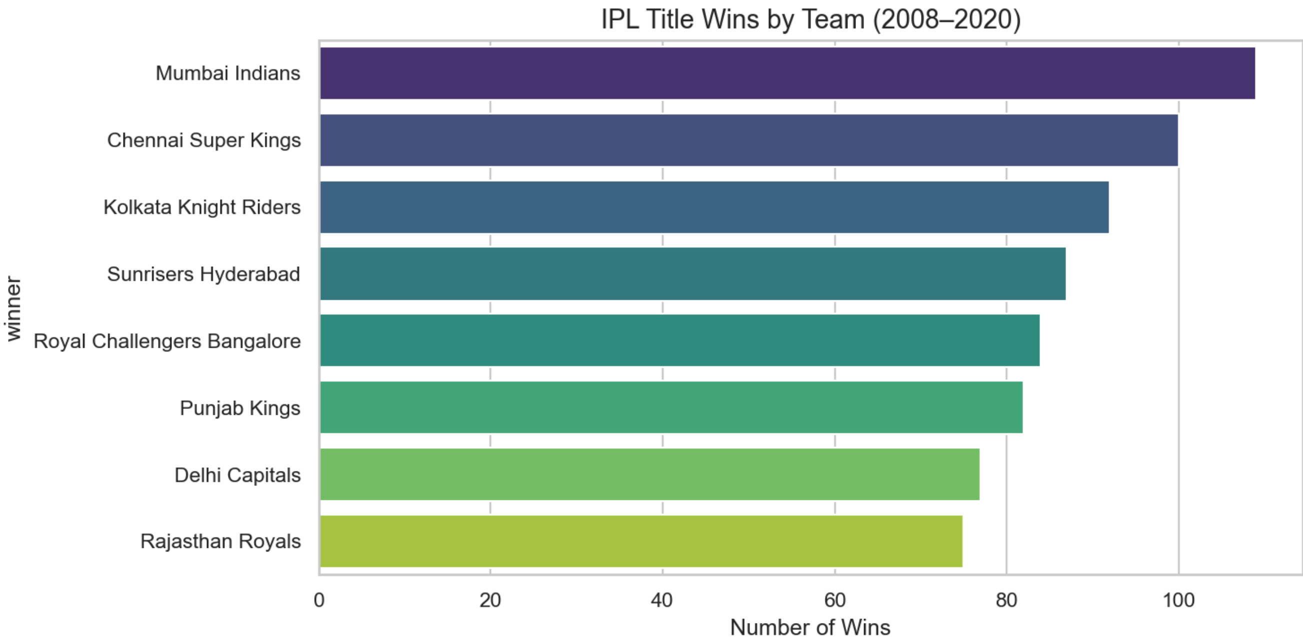


Figure 1 — Total match wins per IPL franchise (2008–2020)

• Mumbai Indians (approx. 109 wins) leads franchises, followed by Chennai Super Kings (100) and KKR (92).
• A large win gap separates the top-3 from the rest
• With long-standing teams Rajasthan Royals and Delhi Capitals remaining in the lower tier.

Figure 2— Total runs by over phase

• Middle overs (7–15) contribute the most runs (≈85K, 43.6%).
• Though Death overs have the highest per-over run rate.
• Powerplay overs generate ≈57K runs setting the tone.

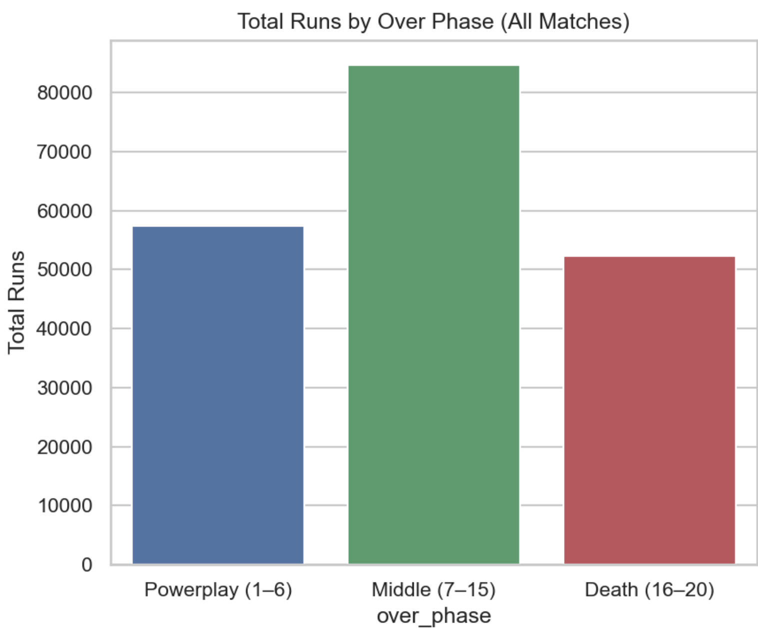


Figure 3: Toss Decision Trend by Season

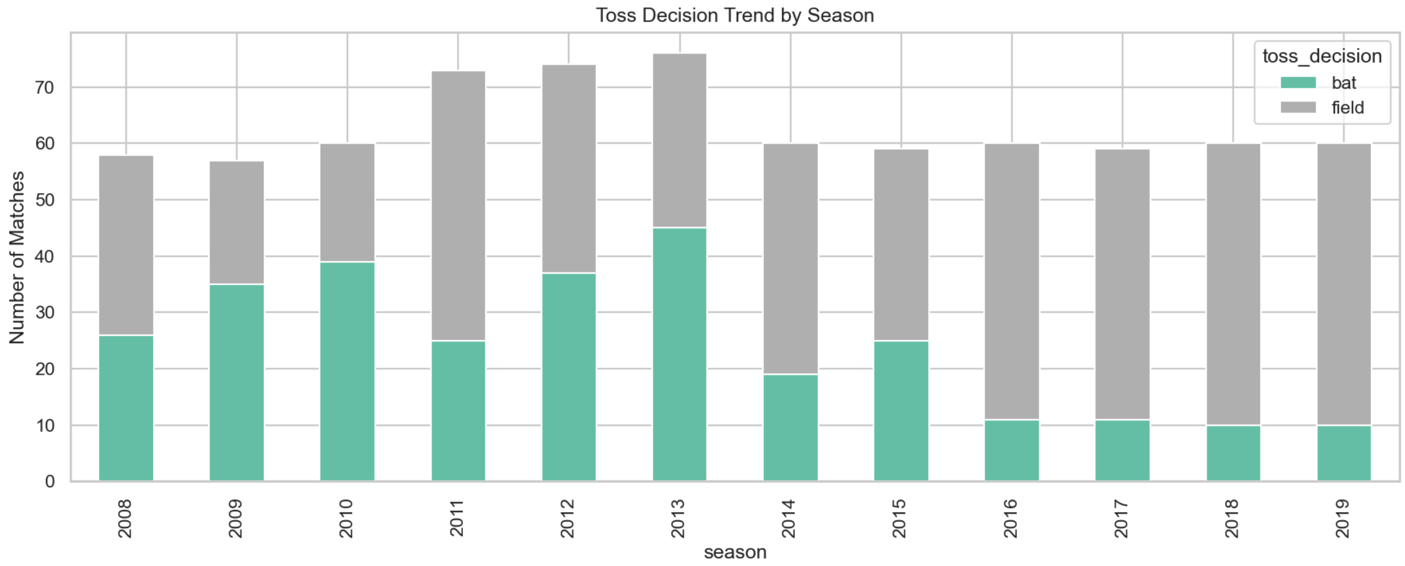
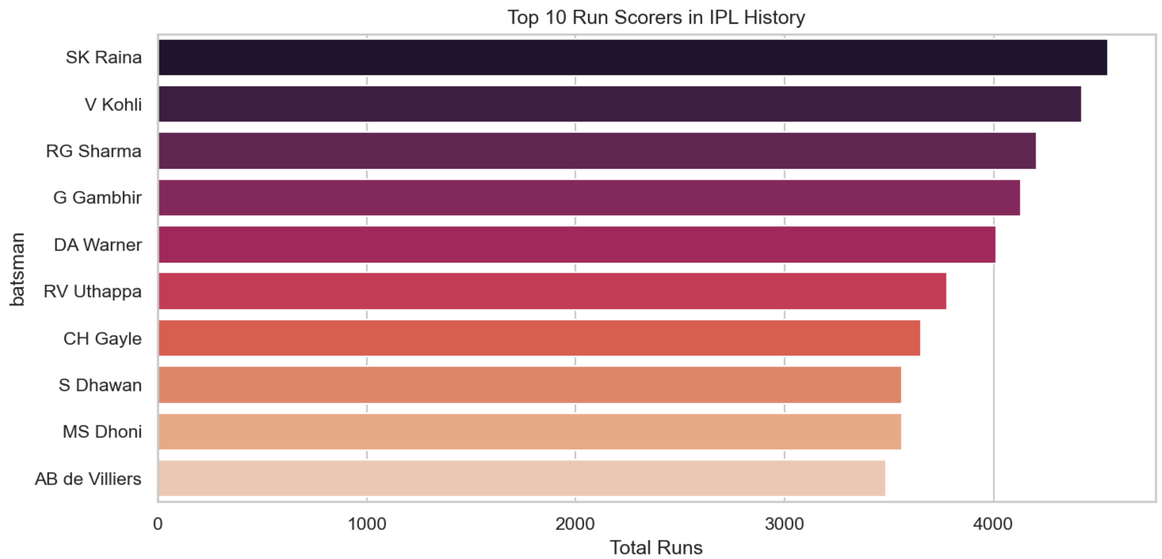


Figure 2 — Toss decision (bat vs field) across IPL seasons 2008–2019.

• Teams in 2008–2013 frequently chose to bat first; from 2014 onward fielding first became dominant.
• This reflects growing T20 awareness of the dew factor and improved chase capabilities.
• 2013 was the peak year for 'bat first' decisions, after which the reversal was decisive.

Figure 4: Top 10 run scorers in IPL history



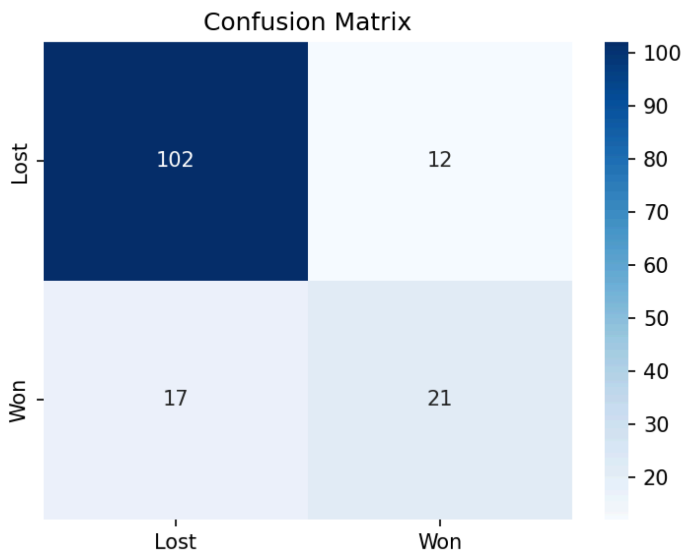
• Top scorers: SK Raina tops (~4,548 runs), followed by V Kohli (~4,423) and RG Sharma (~4,207).

4.2 Model 1: Match Win Prediction — Random Forest Classifier

Table 1: Classification Report

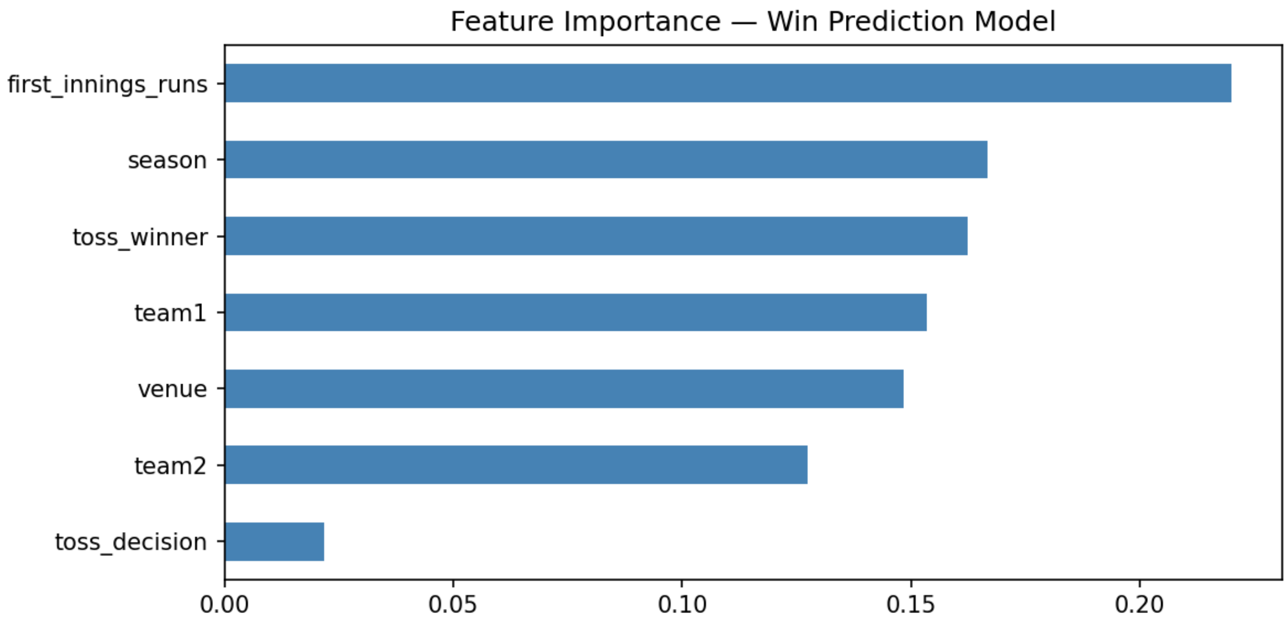
Class	Precision	Recall	F1-Score	Support
0 — Lost	0.86	0.89	0.88	114
1 — Won	0.64	0.55	0.59	38
Macro Avg	**0.75**	**0.72**	**0.73**	**152**
Weighted Avg	**0.80**	**0.81**	**0.80**	**152**
Accuracy			**80.92%**	**152**

Figure 5 — Confusion Matrix



Accuracy: 80.92% (102 losses, 21 wins out of 152 test samples).
Most Important Feature: First innings runs (importance ≈ 0.22).
Moderate Importance: Season and toss winner (~0.16 each).
Minimal Importance: Toss decision (~0.02).
Limitation: Class imbalance (114 losses vs 38 wins) led to low recall for wins; **SMOTE** or class_weight='balanced' recommended.

Figure 6 — Feature Importance Scores



4.3 Model 2: Player Profiling — K-Means Clustering

K-Means clustering was applied to batting statistics of players with at least 20 IPL appearances (n=162 qualifying players). Features: total runs, strike rate, average runs per match, and total sixes. All features were standardised using StandardScaler.

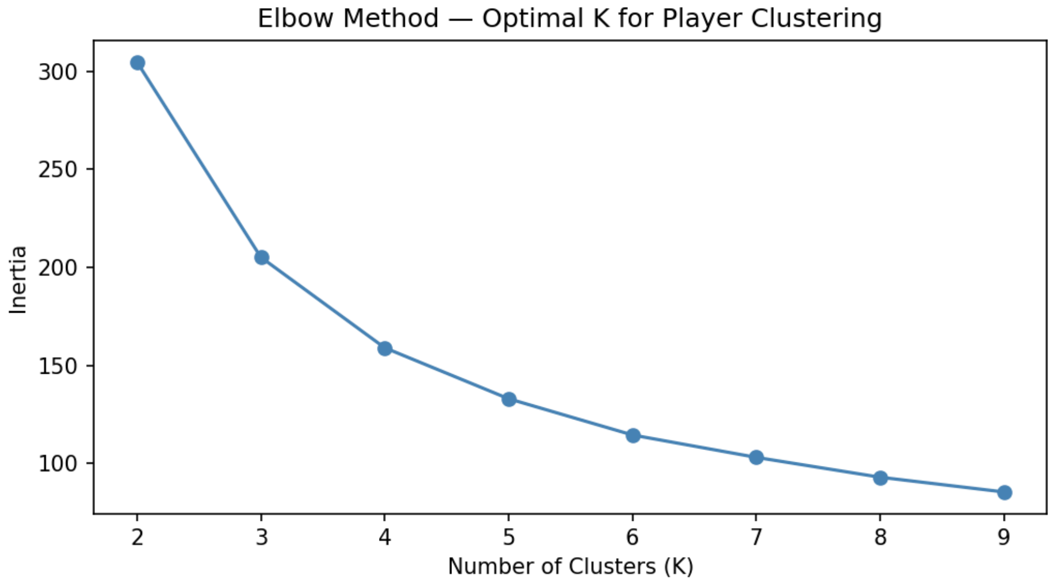


Figure 7 — Elbow Method: Inertia vs Number of Clusters (K)

The elbow is most visible at **K=4**, where inertia decrease transitions from steep to gradual. K=4 was selected as the optimal number of clusters.

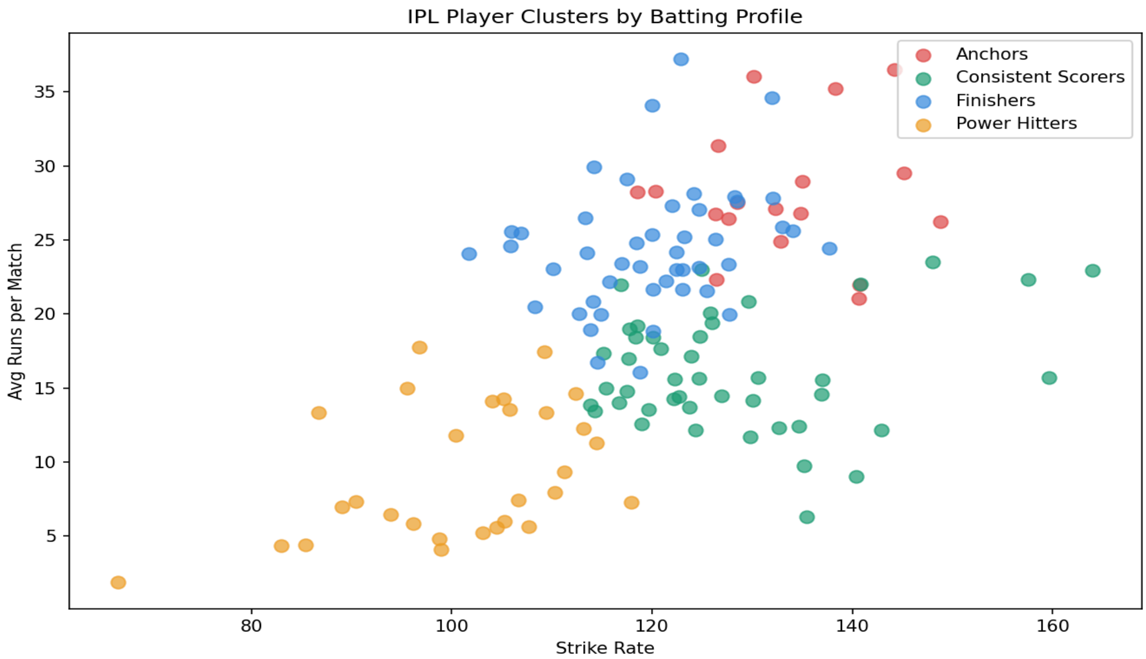


Figure 8 — IPL Player Clusters: Strike Rate vs Average Runs per Match

Table 2: Cluster Descriptions and Top Players

Cluster Profile	Top 3 Players	Role	Profile
Anchors (Red)	SK Raina (4548), V Kohli (4423), RG Sharma (4207)	Top-order run accumulators.	High avg runs, moderate-high strike rate.
Finishers (Blue)	AM Rahane (3057), KD Karthik (2903), M Vijay (2511)	Middle-order / finishing role	Moderate-high strike rate, consistent run contribution.
Consistent Scorers (Green)	DJ Bravo (1262), GJ Maxwell (1229), IK Pathan (1150)	All-rounders & lower-order depth	Moderate strike rate and avg. Reliable contributors.
Power Hitters (Orange)	PP Chawla (515), M Manhas (514), J Botha (409)	Impact players & pinch hitters	Very high strike rate, lower avg per match. Explosive.

Power BI Dashboard

An interactive Power BI dashboard is built from three processed data sources: *matches_cleaned.csv, deliveries_cleaned.csv, and player_clusters.csv*

Figure 9 — Power BI Dashboard: Batting Analysis Page

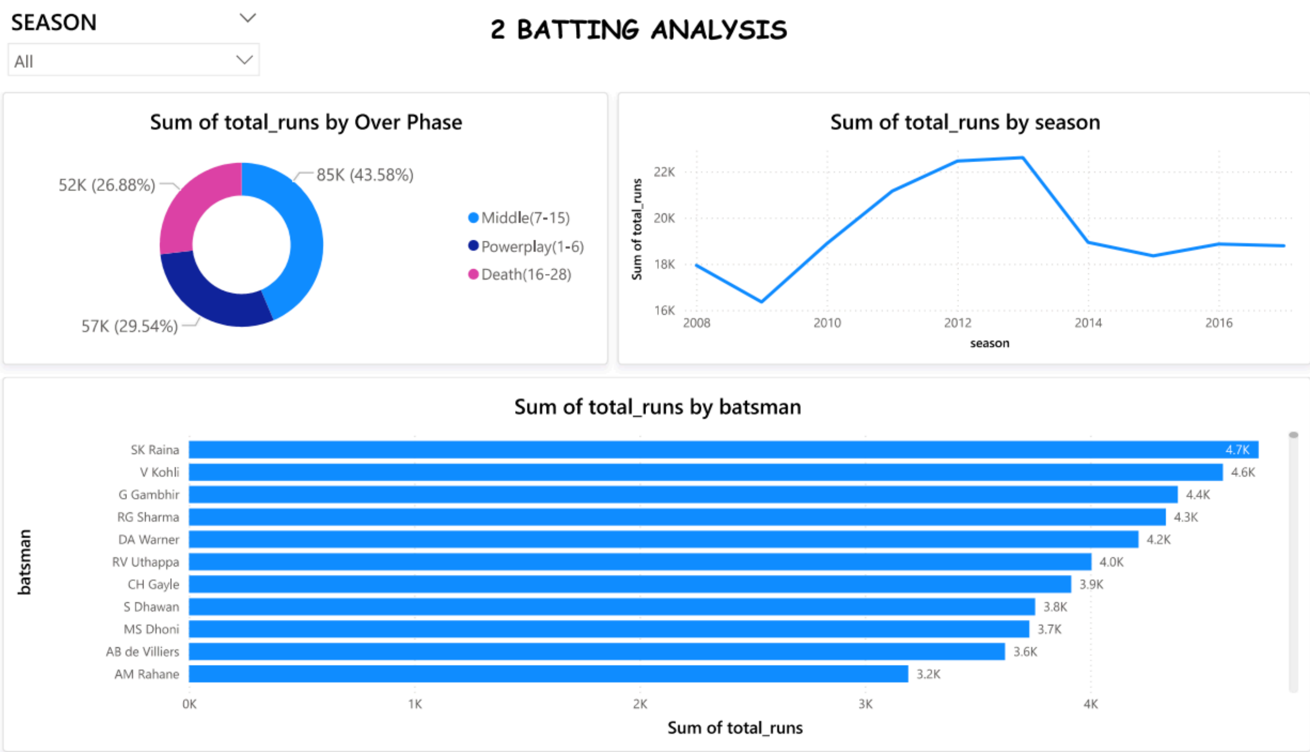


Table 3: Dashboard Pages Summary

Dashboard Pages	Visuals Included
1. Overview	Cards: total matches, seasons. Bar chart: wins by team. Season slicer.
2. Batting Analysis	Donut: runs by over phase (43.6% middle). Line: total runs by season. Bar: top batsmen.
3. Toss & Win Analysis	Pie: toss decision split. Bar: toss advantage per team. Matrix: head-to-head records.
4. Player Clusters	Scatter: strike rate vs avg runs (colored by cluster). Table: top players per cluster.

Key Insights & Recommendations

6.1 Key Insights

• **Mumbai Indians dominate**: With ~109 wins, MI's success correlates with consistently high first-innings scores and strong death-over bowling — both high-importance predictors in our model.
• **Fielding first is now the norm**: Since 2014, over 80% of toss winners chose to field first, reflecting the strategic value of chasing and improved batting depth. • First innings score is the strongest predictor: Feature importance confirms first_innings_runs (importance = 0.22) as the single most predictive variable. Teams scoring 170+ win significantly more often.
• **SK Raina is the all-time leading run-scorer**: With 4,548 runs, Raina's consistency and availability set him apart. Kohli and Rohit follow closely in the Anchors cluster.
• **Middle overs are the run engine**: 43.6% of all IPL runs come from overs 7–15. Teams that accelerate effectively through the middle build winning platforms.
• **Four distinct player archetypes exist :** K-Means cleanly separates batsmen into Anchors, Finishers, Consistent Scorers, and Power Hitters.

6.2 Recommendations

• **Team selection**: Prioritise Anchor-type batsmen in the top 3 and pair them with at least one Power Hitter in the middle order for batting versatility.
• **Toss strategy**: Default to fielding first after winning the toss, especially at venues prone to dew — unless conditions strongly favour batting.
• **Model improvement**: Apply SMOTE oversampling or class_weight='balanced' to address the 3:1 class imbalance and improve recall for the 'Won' class.
• **Phase-specific bowling**: Deploy best economy bowlers in overs 7–15 (highest-run phase) rather than saving them exclusively for death overs.
• **Player auction strategy**: Use the cluster model to identify undervalued Finisher or Consistent Scorer profiles who deliver impact without premium price tags.

Conclusion and Future Scope

7.1 Conclusion

This Datathon project delivered a complete, end-to-end analytics pipeline on real-world IPL data, covering cleaning, feature engineering, exploratory analysis, and model building. The Random Forest classifier achieved 80.92% accuracy for match outcome prediction. K-Means clustering (K=4) identified four meaningful player archetypes.

A Power BI dashboard presents these results interactively. All five Datathon learning outcomes problem framing, tool application, insight generation, visual communication, and teamwork were met.

All project assets (notebooks, data, models, dashboard) are fully version-controlled on GitHub, allowing faculty to track phase-wise progress via commit history.

7.2 Future Scope

- Live data integration: Extend the pipeline to ingest live IPL 2024–25 data via CricAPI for real-time win probability updates.
- Player form tracking: Add a rolling-window feature (last 5 matches) to capture current form rather than only career statistics.
- Bowling analytics: Apply a similar clustering approach to bowlers (economy rate, wicket rate, death-over performance) to build a complete team-building model.
- Deep learning: Explore LSTM or Transformer-based models on ball-by-ball sequences for over-by-over win probability modelling.
- NLP on commentary: Sentiment analysis on match commentary to correlate crowd pressure moments with match outcomes.

References & Appendices

R. References

[1] IPL Complete Dataset 2008–2020. Kaggle (2021). Available at: https://www.kaggle.com/datasets/patrickb1912/ipl-complete-dataset-20082020

[2] Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *JMLR*, 12, 2825–2830.

[3] McKinney, W. (2010). Data Structures for Statistical Computing in Python. In *Proc. 9th SciPy Conf.*

[4] Hunter, J.D. (2007). Matplotlib: 2D Graphics Environment. *Comp. Sci. Eng.*, 9(3), 90–95.

[5] Waskom, M. (2021). Seaborn: Statistical Data Visualization. *J. Open Source Software*, 6(60), 3021.

[6] Microsoft Power BI Desktop (2024). Microsoft Corp. Available at: https://powerbi.microsoft.com.

[7] Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5–32.

[8] MacQueen, J. (1967). Methods for Classification and Analysis of Multivariate Observations. In *Proc. Fifth Berkeley Symposium on Math. Stat. & Prob.*

A. Appendices

Appendix	Description	Location in Repository
A	Data preprocessing notebook	`notebooks/data_preprocessing.ipynb`
B	EDA notebook with all visualisations	`notebooks/EDA.ipynb`
C	Model building & evaluation notebook	`notebooks/modeling.ipynb`
D	Standalone model Python script	`src/model.py`
E	Power BI dashboard file	`dashboard/ipl_dashboard.pbix`
F	Classification report CSV	`outputs/metrics/classification_report.csv`
G	Player cluster results CSV	`outputs/metrics/player_clusters.csv`
H	All EDA and model plots (PNG)	`outputs/plots/`